Worker isolation: test results (branch fix/worker-isolation, base 35884ee)

New tests: server/lib/job-isolation.test.ts

Command:

```
DATABASE_URL=postgres://stub:stub@127.0.0.1:1/stub timeout 600 npx tsx server/lib/job-isolation.test.ts
```

Result (final run, 2026-10-11, code at the branch head): `exit=0`, `# tests 22  # pass 22  # fail 0`, duration 63.0 s. The full log is in `evidence/job-isolation.test.log`.

The test file deletes `REDIS_URL` and `MAX_CONCURRENT_SUBPROC` before its imports. This runs the child with no Redis available and the subprocess limit at its code default.

#	Test	Gap	Measured result
0	flag + PG budget partition	A	C = 4
1	child busy-loops 3 s	G	**parentMaxLag 3.0 ms**, wall 3556 ms
2	cancel	D	“Cancelled” after 2593 ms
3	revoke (ownership_lost), child ignores SIGTERM	D	“Cancelled” after 6489 ms → fenced non-retriable
4	crash holding an LLM permit	D	“Isolated job child exited unexpectedly (code=1 signal=null stage=spawn)”, retriable, permit released
5	permit brokerage	E	maxLlm 8, maxBrowser 1
6	usage forwarding	A	rows in parent buffer, child buffer 0, child PG_POOL_MAX 4
–	ProviderScoringError rebuild	D	real class
7	real 36-measure pipeline in child	G	success, 36 measures, 168 provider calls, wall 12303 ms, **parentMaxLag 67.2 ms**
A1	pool inventory	A	parentMain 4 + children 4×4 + parentUsage 2 + childUsage 0 = **22 = baseline 22**
B1	pipeline deadline, child blocked forever, 30 s grace configured	B	resolved after 2529 ms (limit 2500), “Pipeline timed out after 3s”, revoked `pipeline_timeout`, all descendants gone
B2	job deadline, child dispatching and ignoring SIGTERM	B	“Job watchdog timeout: isolated child killed at job deadline (code=null signal=SIGKILL stage=pipeline)”, non-retriable; last fixture

#	Test	Gap	Measured result
			write deadline+1 ms (tolerance 50 ms), none after
C	supervisor SIGKILL, child synchronously blocked	C	child + grandchild + detached descendant gone after 41 ms
C	supervisor SIGKILL, child idle	C	gone after 10 ms
D	retry classification table (8 rows)	D	all rows match the in-process outcome
D6	cancel in progress + job deadline	B/D	“Cancelled” after 2510 ms (killed at the deadline, not after the 30 s grace)
E1	queued IPC LLM acquire aborted while 8 held	E	pending 0 after abort, sawQueued and sawWithdrawn true
E1b	child dies with 8 held + 1 queued	E	all 8 returned, queued wait dropped
E2/E3	child reports success but leaves a grandchild + permit alive	E	delivered after 3517 ms; at delivery child/grandchild/proc/live = false/false/0/0; releasedWhileAlive false
E4	3 children × 4 subprocess permits	E	maxProc **6**
F1	child startup audit	F	handles `PipeWrap, Timeout` at entry and after the pipeline import (IPC pipe only); 112 modules; server modules only `storage.ts`, `db.ts`; no redis/bullmq/startup-cleanup/reconciler; pgTotal 0; usagePool false
G1	4 CPU-bound children on 2 CPUs	G	**parentMaxLag 10.4 ms**, forks 4, maxForkMs 7.6, messages 12, maxHandlerMs 0.10

parentMaxLag across the three full runs on this code:

Run	Test 1	Test 7	G1	Note
run 2	3.8 ms	7.7 ms	4.5 ms	
run 3	1.6 ms	10.2 ms	4.6 ms	
run 4 (final)	3.0 ms	67.2 ms	10.4 ms	after the deadline-cuts-exit-grace change

Every run used exit 0 and passed 22/22. Run 1 failed only F1, because of an over-strict socket assertion; that has been fixed.

Regression suites (each run with `npx tsx <file>`, stub DATABASE_URL; all exit 0)

Suite	Result
lifecycle-fence.unit	42/42
scoring-cancellation.unit	60/60

Suite	Result
credit-breaker	22/22
llm-usage	20/20
adaptive-concurrency	13/13
runtime-diag.unit	21/21
reconcile-resurrect	15/15
reliability	PASS
provider-resilience	112/112
transport-abort.unit	55/55

These suites ran before the final one-line `onDeadline` change in `job-isolation.ts`. That module is imported only by `worker.ts` behind the gate and by `job-isolation.test.ts`, so the suites were not re-run.

tsc

`npx tsc --noEmit -p .` gives 62 errors on the branch and 62 at HEAD `46c42ff` (changes stashed). The sorted diff of the error lists is empty, so this branch adds no new type errors. It was re-run after the final change: still 62, none in the isolation, retry-policy or child files.

Not verified

- No runs against real Postgres, Redis or Railway. All tests use a stub PG URL (port 1) and no Redis.
- **No real Puppeteer/Chromium launch in a child.** Descendant-group registration (`processor.ts` → `pgroup`) was exercised with a detached `sleep` in the fixture, not with Chromium.
- Production deadline values (pipeline 35 min, job 40 min) were not exercised. The tests use the same code paths with second-scale values.
- `WORKER_ISOLATION_ENABLED` stays **off** (the default). Staging is still needed for memory headroom (~200 MB per child, up to 4 Chromium instances per replica), real DB latency under the per-child cap of 4, and production CPU contention.
- No paid or real batch runs were made.